

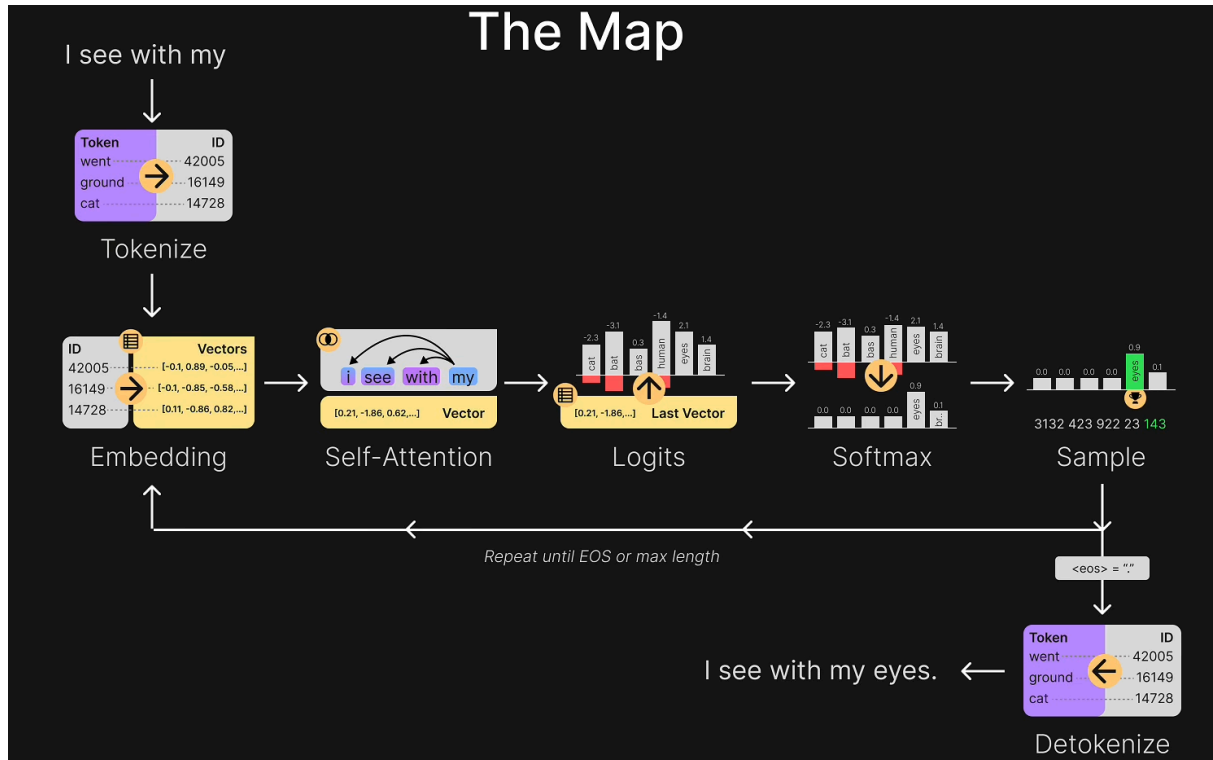
1.LLM ve Embedding Temelleri

1.1 LLM'in Çalışma Mantığı:

LLM (Açılımı 'Large Language Model'dır ve Türkçe de 'Büyük Dil Modelleri' olarak da bilinir), muazzam miktarda metin üzerinde eğitilmiş, doğal dili ve diğer içerik türlerini anlayabilen ve çok çeşitli görevleri yerine getirebilen bir derin öğrenme modeli kategorisidir. Daha detaylı tanım yapmak istersek de bir dizide bir sonraki kelimeyi tekrar tekrar tahmin edebilen devasa bir istatistiksel tahmin makinesine de benzetebiliriz.

Ayrıca LLM'ler insan tarihi açısından da önemli bir yere sahiptir. Çünkü, düzenlenmemiş ('unstructured') insan dilini büyük ölçekte işleyebilen ve makineler ile insanlar arasındaki doğal iletişimi mümkün kılan ilk yapay zeka sistemleridir.

Bu tanımlardan ve bilgilerden sonra temel olarak nasıl çalışır, arkasındaki mantık nedir ona bakmamız gerekir. LLM'in bir metni anlayıp cevap üretebilmesi için geçtiği süreçler ise şunlardır:



(Bkz: Temel çalışma mantığını gösteren diyagram)

- 1) Tokenization: Önce metnimiz “token” adı verilen makinenin anlayabileceği küçük veri parçalarına böler. Bu durum sayesinde dil standartlaşarak yeni ve nadir kelimelerin tutarlı bir şekilde ele alınmasını sağlar (Token; Kelimeler, alt kelimeler veya karakterler gibi daha küçük birimlerdir. 1 token ortalama olarak 4-5 karakterdir ancak bu sayı dile ve diğer çeşitli değişkenlere göre artabilir). Ve her token bir ID alır.



(Bkz: Örnek bir tokenizasyon ve ID haritası)

- 2) Embedding: Embedding, her bir token'ı çok boyutlu uzayda vektör olarak temsil etmeye denir. Tokenization sonrasında bulduğumuz bu ID'leri ise Embedding Matrix'ine çok boyutlu bir koordinat sistemine vektör olarak aktarılır. Bu vektörler ise örnek bakımından “Kral - Erkek + Kadın = Kraliçe” mantığını kurabilmesini sağlar. Yani ‘Kral’ kelimesindeki erkeklik yönünü çıkartıp kadınlık yönünün eklendiğinde bunun uzay vektöründe ‘Kraliçe’yi gösterebilmesidir.

Token	ID	Embedding vector (dim=6)
went	42005	[+0.55 +0.34 -0.80 -0.29 -0.06 +0.07]
to	7396	[+0.59 -0.88 +0.52 +0.73 -0.82 +0.80]
in	1739	[+0.04 -0.16 -0.56 -0.74 -0.03 +0.25]
the	48698	[-0.01 -0.28 +0.44 -0.11 -0.09 -0.95]
outside	18124	[-0.16 +0.95 +0.23 +0.39 +0.02 -0.62]
ground	16149	[-0.03 +0.80 +0.07 +0.75 +0.15 +0.99]
cat	14728	[-0.31 -0.54 -0.98 +0.86 -0.35 -1.00]
sat	9244	[+0.71 +0.60 +0.19 +0.77 +0.48 +0.62]
plant	48365	[+0.04 +0.92 -0.32 +0.42 +0.85 -0.90]
on	6817	[-0.64 -0.03 -0.84 +0.05 +0.84 +0.75]
mat	44448	[+0.34 +0.71 -0.51 -0.83 -0.13 -0.67]
tree	48640	[-0.27 -0.94 -0.65 -0.12 +0.57 -0.11]
.	35841	[-0.32 -0.71 -0.58 -0.94 -0.49 +0.70]
we	5797	[+0.81 +0.53 +0.38 +0.45 +0.67 +0.40]
are	38798	[+0.03 -0.52 -0.93 -0.14 +0.95 +0.24]
i	27751	[-0.79 -0.40 +0.87 -0.03 +0.44 +0.52]
learning	2182	[-0.27 -0.22 -0.30 +0.12 +0.29 +0.31]
a	2052	[-0.04 +0.27 -0.62 -0.81 -0.38 -0.42]
house	6240	[-0.21 +0.54 +0.42 +0.35 +0.57 -0.37]

(Bkz: 6 boyutlu bir ‘Embedding’ matrisi. Token, ID, Vector eşleşmesi).

- 3) Self Attention: Self-Attention yani Öz-Dikkat mekanizması, en basit olarak verilen metinde kelimelerin ve belirteçlerin önemini tartmak ve aralarındaki ilişkileri daha iyi anlamak için kullanılır. Örnek vermek gerekir ise “ The Animal didn’t cross the road because it was too tired.” cümlesinde self-attention mekanizması sayesinde ‘it’ kelimesinin ‘road’ kelimesine değil de ‘animal’ kelimesine referans yaptığını anlayabiliriz.

LLM’in çalışma mekanizmasında ise her kelimeye şu 3 soruyu sorarak yeni bir vektör yaratır ve bu sayede hangi kelimeler birbirleri ile daha ilişkili onu anlar. Sorularımız ise şunlardır:

Query: “Ben ne arıyorum”, Key: “Diğer kelimeler ile nasıl eşleşirim?”,
Value: “Seçilirse hangi anlamı vereceğim?”

Mekanizma, bir kelimenin ‘Query’ vektörü ile diğer kelimelerin ‘Key’ vektörlerini çarpar; Çıkan sonuç yüksekse o 2 kelime birbirleri ile bağlantılı demektir. Çarpım sonucu çıkan bu vektör de ‘Value’ vektörü ile çarpılarak asıl kelimenin anlamı zenginleşir.

- 4) Logits: Sinir ağının son katmanında üretilen ve olasılıklara dönüşmeden önce context vektörlerinden edilen ham, normalleştirilmemiş skorları temsil eder.
- 5) Softmax fonksiyonu ile logits’ten ham skorları pozitif olasılık değerlerine dönüştürür. Bu sayede sözlükteki tüm kelimeler için bir olasılık puanı çıkartılır. Sampling ile de olasılık puanı en yüksek olan kelime seçilir.
- 6) Detokenization: En son olarak token’lar sayı olarak değil de insanların okuyabileceği dil biçimine çevirerek ekrana yazdırılır

1.2 Token ve Context Window Nedir?

Token; Kelimeler, alt kelimeler veyahut karakterler gibi daha küçük birimlerdir. 1 token ortalama olarak 4-5 karakterdir ancak bu sayı dile ve diğer çeşitli değişkenlere göre artabilir.

Token'lara ihtiyacımız olmasının en temel sebebi dil modellerinin metinleri okuması amacıdır.

Context Window: LLM'in context window'u, herhangi bir anda dikkate alabileceği ya da hatırlayabileceği metin miktarıdır. Context Window'un daha büyük olması demek; AI modelinin daha fazla input alabilmesi ve her çıktıya daha fazla bilgi dahil edilmesini sağlar.

Context Window aşıldığında veyahut sınırlarına ulaşıldığında 'AI Hallucination' denen bir kavramın yaşanma ihtimalini artırır. AI Hallucination ise kısaca AI'nın halüsinasyon görüp hiç olmayan gerçekleri sanki varmış gibi söylemesidir.

AI ile konuşurken ne zaman AI'nın ne zaman halüsilasyon gördüğünü tespit edip ilgili sohbeti bırakmak için de garip taktikler oluşturulmuştur. Son zamanlarda popüler olan bir örnek aşağıda bulunmaktadır.



Zach O'Brien • 3.+

Niche Publisher. Social Media Marketer. Writer. Podcast/Video ...

1 ay •

+ Takip Et ...

The longer your Claude chat runs, the dumber it gets. Here's how to know when it's getting dumb.

As the context window fills up, Claude quietly starts dropping your instructions to save room. Your rules just stop getting followed.

The trick I use to catch it: open your instructions file and add one line:

"Call me Zach every time you reply."

Obviously, use your own name (unless your name is also an awesome name like Zach). Now every response starts with it.

The moment Claude stops saying my name, that's my signal that context is getting full and my rules are slipping with it.

When the name disappears, compact the conversation or start fresh. My rule of thumb is to compact around 50% so you're always working with Claude at its sharpest.

1.3 Embedding Nedir?

Metin, görüntü ve ses gibi nesneleri, makine öğrenimi algoritmaları için anlamsal olarak anlamlı olan sürekli bir vektör uzayındaki noktalar olarak temsil etme yöntemidir.

Embedding işlemi, her kelimeyi çok boyutlu bir uzayda belirli bir koordinata vektör olarak yerleştirir. Bu yerleştirme rastgele yapılmaz: Anlamca birbirine benzeyen veya birbiriyle ilişkili kelimeler, bu uzayda birbirine yakın konumlarda bulunur. “Kral - Erkek + Kadın = Kraliçe” mantığını kurabilmesini sağlar. Yani ‘Kral’ kelimesindeki erkeklik yönünü çıkartıp kadınlık yönünün eklendiğinde bunun uzay vektöründe ‘Kraliçe’yi gösterebilmesidir.

Metin ve görüntü arama motorları öneri sistemleri, dolandırıcılık tespit sistemleri gibi birçok uygulama geliştiren mühendisler için kritik bir araçtır. Temel olarak makine öğrenimi modellerinin benzer nesneleri bulmalarını sağlar.

Diğer makine öğrenimi tekniklerinden farklı olarak tanımlanması için insan uzmanlığı gerektirmez. Sinir Ağları gibi çeşitli algoritmaları kullanılarak verilerden öğrenilir ve bu sayede model, insanların tespit etmesi mümkün olmayan karmaşık kalıpları ve ilişkileri verilerden öğrenebilir.

1.4 Cosine Similarity Nedir?

Kosinüs benzerliği, 2 veri noktasının uzunluk ve boyutlarından ziyade işaret ettikleri yöne göre ne kadar benzer olduklarını belirleyen bir ölçüttür. Yüksek boyutu uzaylarda etkilidir.

Kosinüs benzerliğini hesaplamak için 2 vektör arasındaki açının kosinüsünü ölçmek gerekir. Bu ölçüm sonucunda ise kosinüs benzerlik puanı üretilir. Bu değerler -1 ile 1 arasında değişir.

Benzerlik puanı 1 ise aynı yöne,
Benzerlik puanı 0 ise yönsel olarak benzer olmadıkları,
Benzerlik puanı -1 ise zıt yönde oldukları bilgilerini ediniriz.

Peki bu kosinüs benzerliği ne işe yarar? Bu sorunun cevabı ise embedding ile bağlantılıdır. Embedding ile metindeki her bir kelimeyi uzayda vektör olarak yerleştirir demiştik. Kosinüs benzerliği ile de bu vektörlerin açısı karşılaştırılır ve açısı en dar olan seçilerek en benzer olan yani en alakalı olan bulunur.

Modern Sistemlerde de oldukça kullanılır. Spotify’da şarkı dinlediğinizde spotify , dinlediğiniz şarkının vektörü ile veritabanlarındaki tüm şarkıların vektörleri ile kıyaslayıp açısı en dar olan yani en benzer olan şarkıyı bulup size tavsiye etmesi bu sistemlere örnektir.

Kaynakça:

<https://www.ibm.com/think/topics/self-attention>

<https://www.ibm.com/think/topics/embedding>

<https://www.ibm.com/think/topics/large-language-models>

How LLMs Work: A Visual Guide:

<https://www.youtube.com/watch?v=PaNPWuASOvM&t=97s>

https://en.wikipedia.org/wiki/Large_language_model

<https://www.databricks.com/blog/what-are-large-language-models>

https://www.reddit.com/r/NovelAi/comments/nohvse/can_someone_explain_what_exactly_is_an_ai_token/

<https://www.datacamp.com/blog/self-attention>

<https://www.ibm.com/think/topics/context-window>

Machine Learning Crash Course: Embeddings:

<https://www.youtube.com/watch?v=my5wFNQpFO0>

Cosine Similarity Explained: How Spotify & Netflix Recommend What You Love!:

<https://www.youtube.com/watch?v=kJeqX7c6pzg>

<https://www.ibm.com/think/topics/cosine-similarity>